Global Capacity Orchestrator

One API. Every Accelerator. Any Region

Last updated 05/29/2026

Who am I?

- Background in bioinformatics and machine learning; managed large-scale DevOps for bioinformatics analysis at a synthetic biology company prior to coming to AWS. Also was involved in the production of the Moderna mRNA vaccine for Covid-19 during the pandemic (was categorized an essential worker).
- I'm the creator of the following projects:
  - Global Capacity Orchestrator on AWS (the topic of this presentation)
  - The bioinformatics workflow framework Shepard (blog / code)
  - The infrastructure library Morphogen2 (blog)
  - Microservices framework Fluoride (blog / code)
  - OpenEMR on EKS (code), OpenMRS on ECS (code) , OpenEMR static Binary Forge (code; allows compilation of mostlystatic PHP interpreters for use with OpenEMR for FreeBSD, Linux, and macOS), and multiple contributions to the source code of OpenEMR and almost all of the code for the current production v8.1.0 version of OpenEMR found in dockerhub
  - OpenMRS on ECS (code) and multiple contributions to the source code of OpenMRS (successfully advocated for changes to the backend to enable horizontal scaling which became a roadmap item)
- While at AWS:
  - Though Leadership Champion award winner from the Healthcare and Life Sciences Technical Field Community for 2025 for work with open-source electronic health records on 6 continents.
  - 4 x finalist for HPCWire's 2025 Editor's/People's choice awards (including 2 different nominations for "Best Use of HPC in Life Sciences".
  - Led AWS's collaboration with Columbia University and Novo Nordisk in making Openfold3; the most advanced open-source protein folding model (blog)
  - Led AWS's collaboration with Harvard Medical School in democratizing access to structural biology tools critical for drug discovery (blog)
  - Led AWS's collaboration with NASA and Navteca in creating and open-sourcing "Open Science Studio" (blog; first author; try for free at https://smce.nasa.gov/oss/)
  - Part of AWS's collaboration with Stanford University's Artificial Intelligence in Medical Imaging Institute where we developed new techniques for privacy-preserving federated learning and then applied those techniques to create an AI model that diagnoses and segments pediatric brain tumors. (publication/code)
    - This work was published in Nature Communications and then featured Nature's "Applications of Artificial Intelligence in Cancer" collection for "significantly [moving] forward the rapidly evolving field of cancer research" and has been cited over 30 times and was the topic of the my presentation for the 20th SBRC International Cryo-EM Seminar entitled "*Using Federated Learning and AI to Treat Rare Disease and Cancer*" hosted by "The High Energy Accelerator Research Organization" in Japan.

LinkedIn
GitHub
https://www.jacobmevorach.com/

Agenda

01	**Solution Overview** Architecture: GA → API Gateway → EKS Auto Mode	10 min
02	**Auto Routing** Getting workloads where they need to be automatically	10 min
03	**GCO MCP** Running jobs and deploying inference using nothing but natural language	10 min
04	**Live Demo & FAQ** CLI deployment, job submission, autoscaling	10 min

01

Solution Overview

Addressing GPU capacity constraints & multi-region complexity

The Problem: Finding and best utilizing GPU Capacity

What customers are telling us:

- *"We can't get GPU capacity in our primary region"*
- *"Managing multi-region infra is a nightmare"*
- *"Our teams spend more time on infrastructure than ML"*
- *"How can I serve multi-region inference with automatic failover?"*
- *“How can I scale my workload?”*

GPU demand is outpacing supply and there’s capacity out there that customers don’t know about or can’t use easily that can help us meet this challenge.

What could be accomplished if we always knew how to find capacity and, when we did find it, how to immediately put it to use?

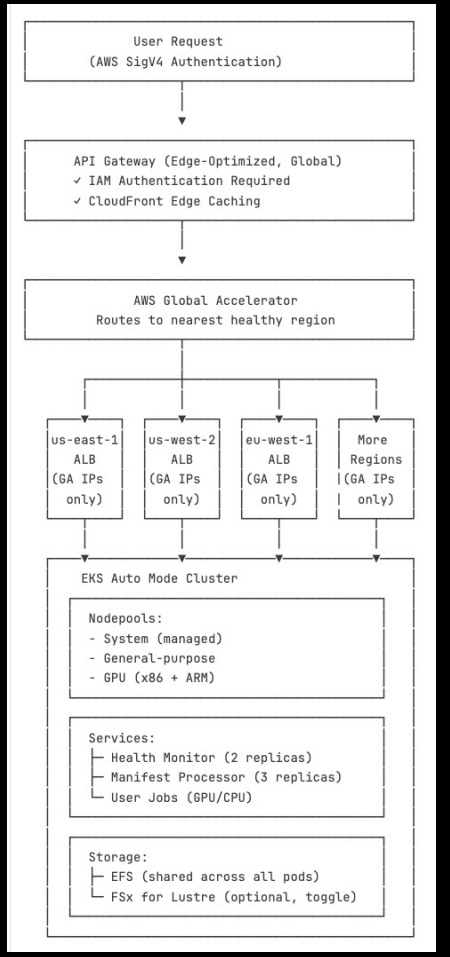
The Solution: Uber but for compute

How much better would life be if we knew exactly where capacity was available and could use it immediately?

Challenge	Traditional Approach	With Global Capacity Orchestrator
GPU Availability	Manually check each region	Auto-routes to available capacity
Node Provisioning	Pre-provision or wait for scaling	EKS Auto Mode provisions on-demand
Multi-Region Ops	Manage clusters separately	Single API, automatic routing
Authentication	Configure per-cluster kubeconfig	IAM-based, existing AWS credentials
Job Outputs	Lost when pods terminate	Persisted to EFS / FSx storage
Failover	Manual intervention required	Automatic via Global Accelerator

What is Global Capacity Orchestrator?

- Global Capacity Orchestrator is a multi-region compute orchestration platform
- You get the complete source code that consists of …
  - Infrastructure written in pure CDK
  - A well documented API that uses IAM authentication
  - An accompanying CLI that makes using the API easy
- Capable of scaling compute across 38+ regions to over 27 million GPUs (theoretical maximum; in practice you'll be limited by the actual compute available at any one time)
- Capable of running complex worfklows, distributed training and above all inference in the form of multi-region, distributed inference endpoints with automatic failover and centralized management.
- All metrics from every region flow into a single CloudWatch dashboard so monitoring can be done from a single pane of glass.
- All infrastructure is bundled into a single CDK deployment orchestrated by the GCO CLI



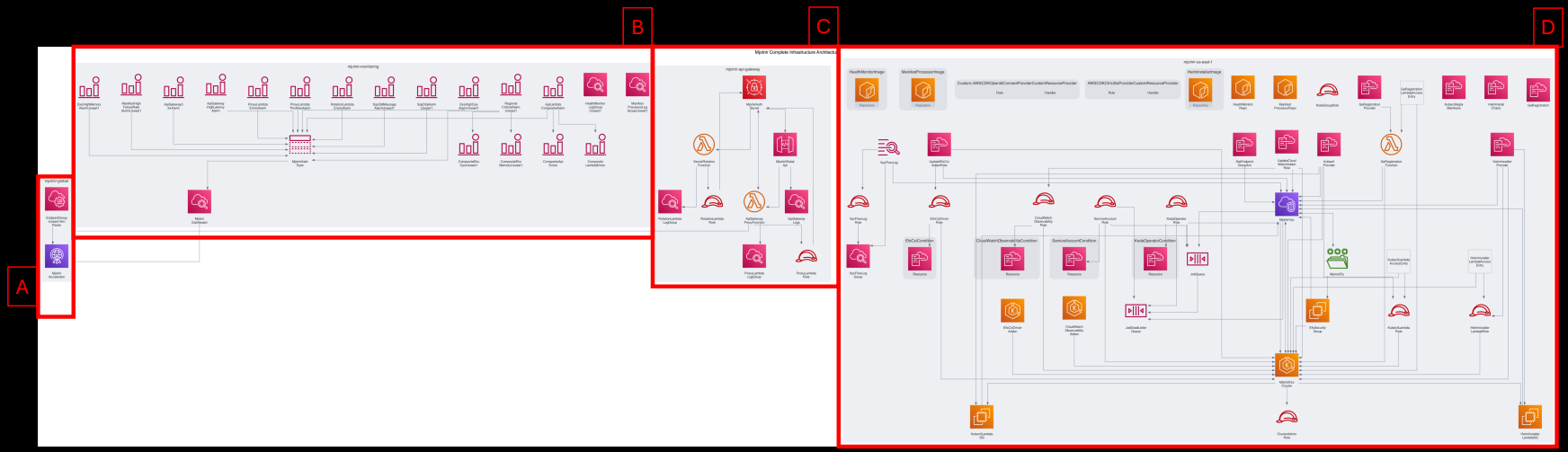
Deploying with a single command!

- Set up the GCO CLI
  - Set up AWS CDK
  - Set up your preferred container runtime (will auto-detect and support Docker, Finch, and Podman).
  - Install Python CLI with pip
  - (optional) Install fastmcp with pip to use the MCP

- Run a single command: “gco stacks deploy-all -y”

- Wait <60 minutes (even true for multi-region deployments when done with the ”--parallel” flag)

Programmatically Generated Diagrams

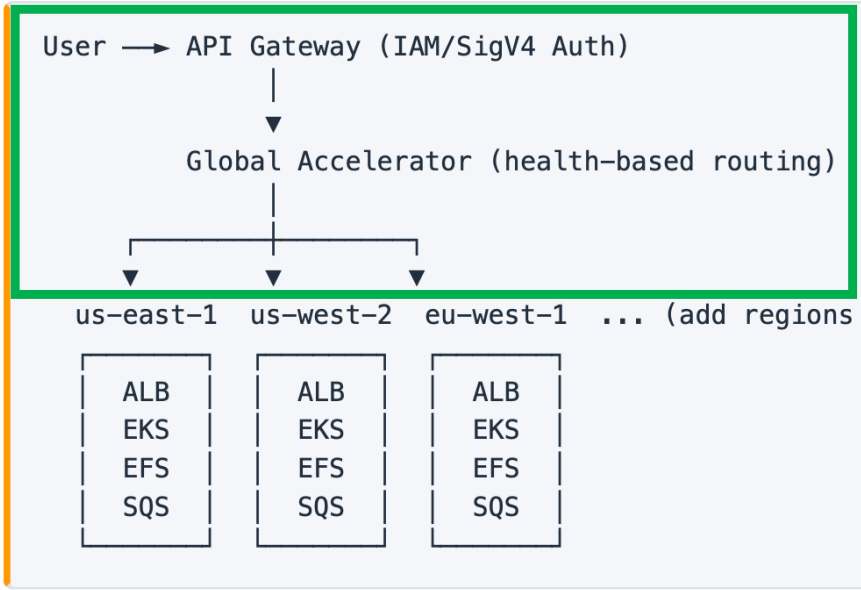


Note: This diagram was autogenerated using AWS-PDK (use generate.py in the “diagrams” folder at any time to generate these diagrams programmatically from the CDK code)

A. Global Stack – Global Accelerator, cross-region routing
B. Monitoring stack – CloudWatch dashboards, alarms, SNS
C. API Gateway Stack – REST API with IAM auth, WAF, Lambda proxy
D. Regional Stack – EKS auto mode cluster with ML frameworks pre-installed, ALB, SQS, EFS per region

Regional stack is replicated in each specified region!

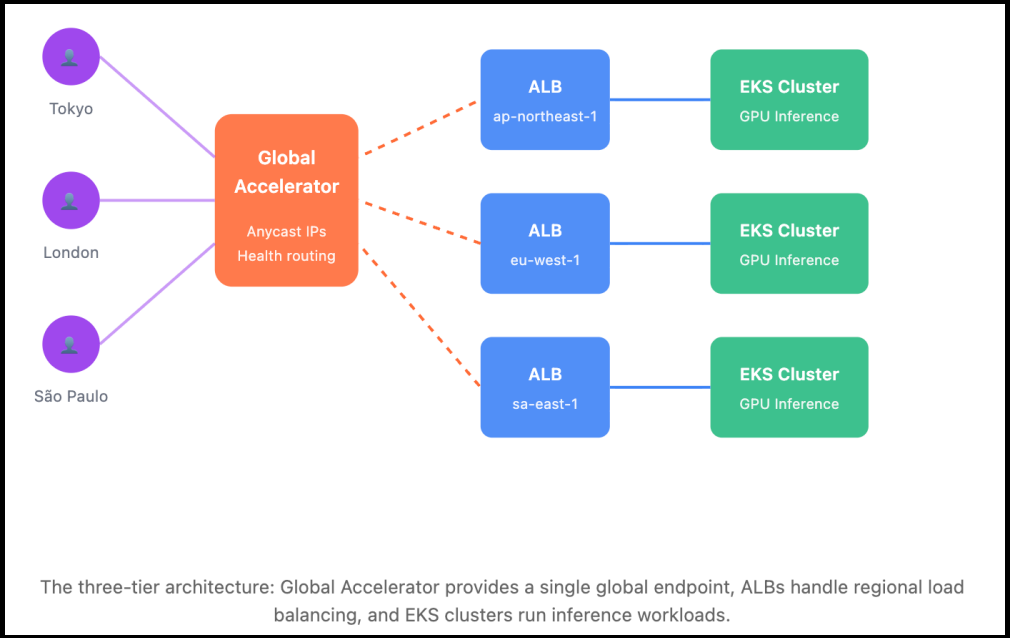
Maintaining end-to-end edge acceleration for inference and batch jobs



Each region runs an independent EKS Auto Mode cluster with GPU nodepools, shared storage, health monitoring, SQS job queues, and manifest processing. Adding a region is a one-line config change and a redeploy.

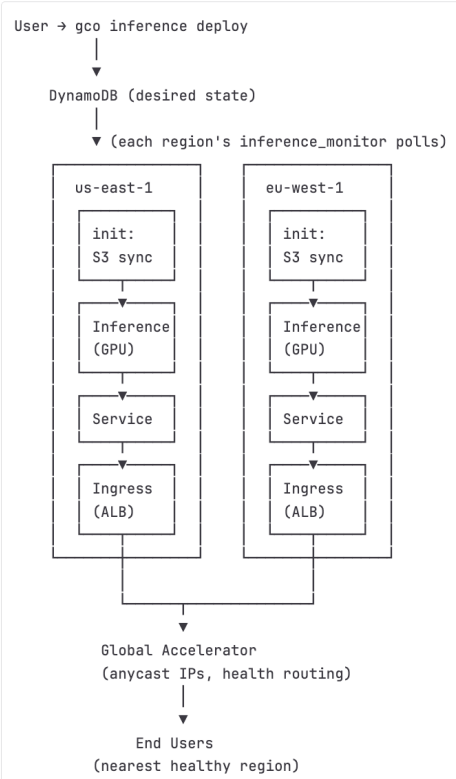
Inference Architecture

https://day1inference.com/global-inference-serving/



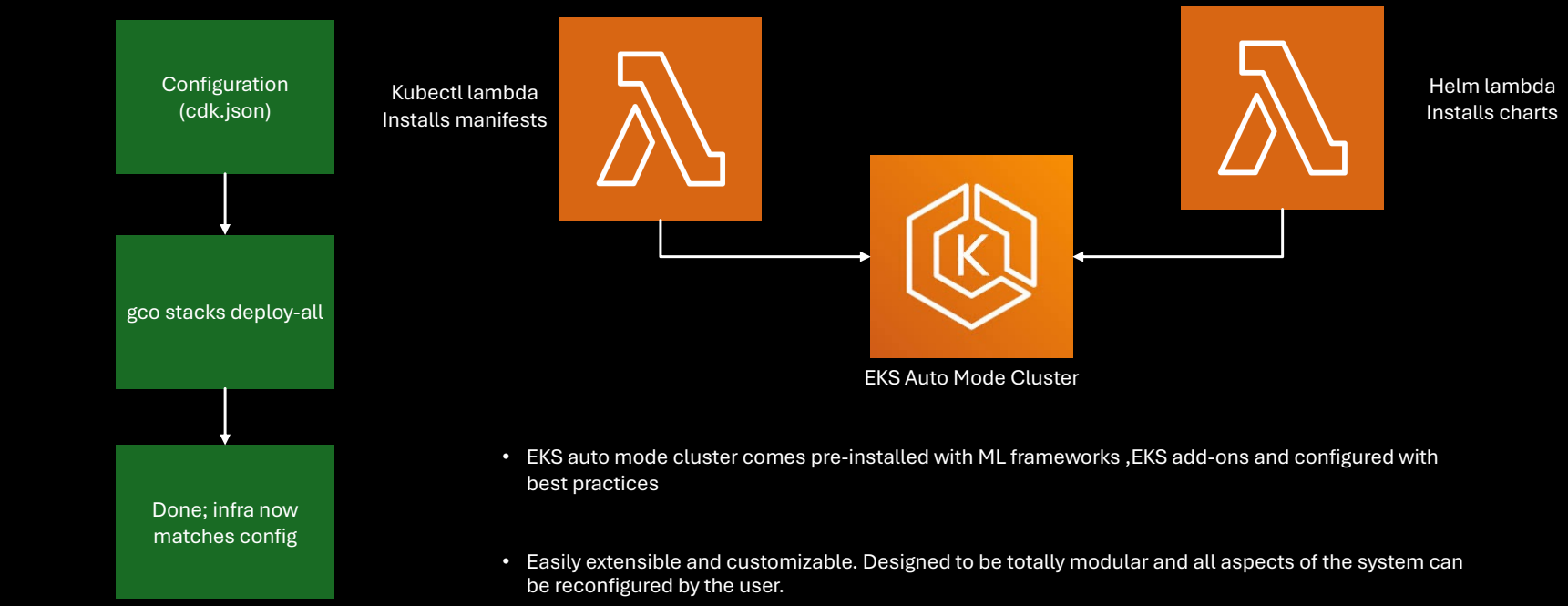
The three-tier architecture: Global Accelerator provides a single global endpoint, ALBs handle regional load balancing, and EKS clusters run inference workloads.

Inference serving uses a reconciliation pattern similar to Kubernetes controllers:



The Problem: Operational Complexity

The Solution: Idempotent Deploys and Managed Cluster Setup



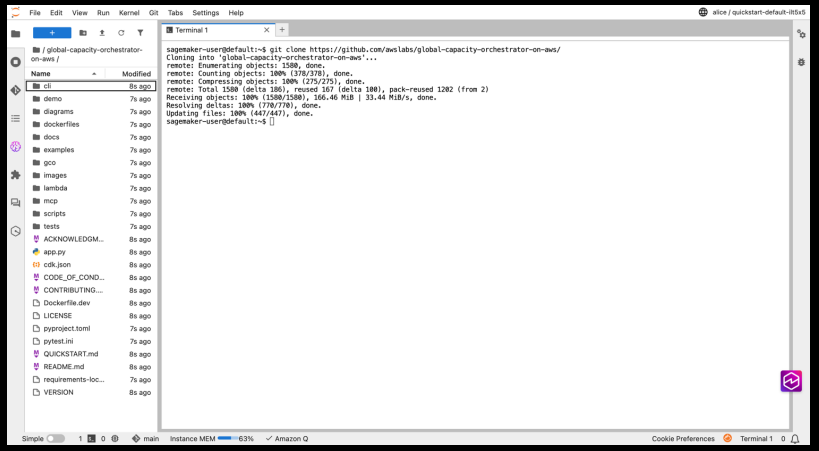
Idempotent Infra Management
One file and one command

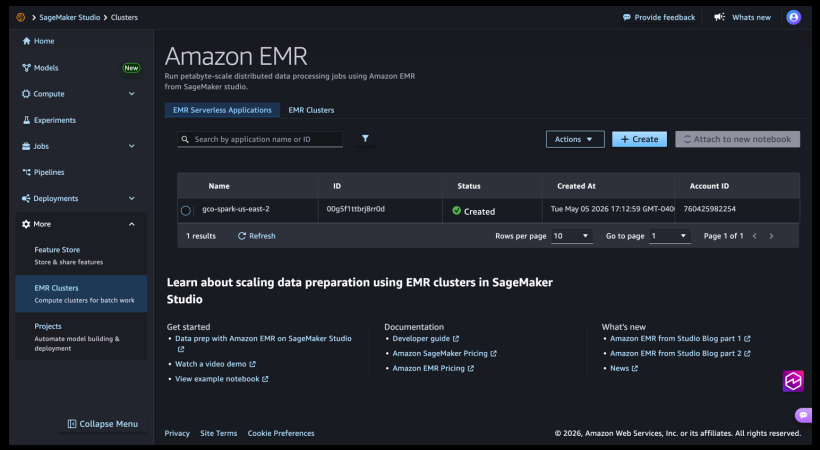
- EKS auto mode cluster comes pre-installed with ML frameworks ,EKS add-ons and configured with best practices
- Easily extensible and customizable. Designed to be totally modular and all aspects of the system can be reconfigured by the user.

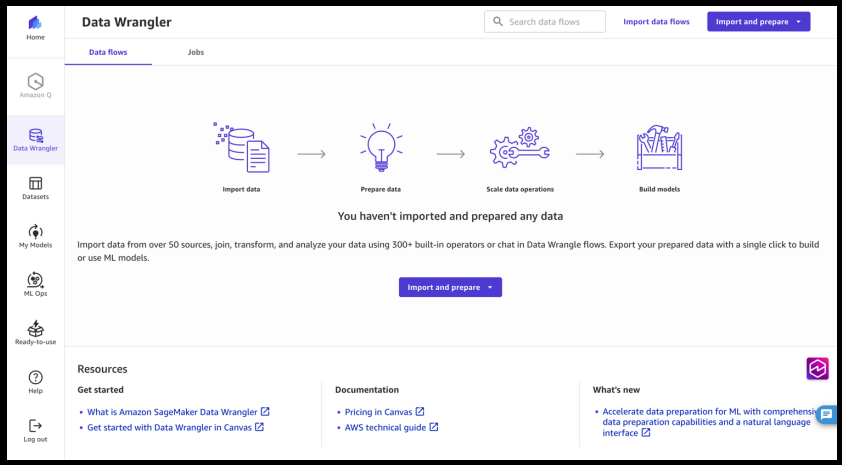
Integrated Analytics Environment

- Automated user management with Cognito and GCO CLI
- Encrypted SageMaker environment with KMS key
- EFS for persistent storage
- EMR Serverless cluster for data analysis and Spark jobs
- Shared JupyterLab environment with persistent storage
- Access a shared S3 bucket that all clusters can also read/write to
- Submit to GCO clusters in multiple regions from the environment
- Canvas for no-code ML and data processing
- MLFlow for experiment tracking!
- Deploy with a single command!

- For a complete guide see ANALYTICS.md in the “docs” folder!







Webhooks and Templates

Globally accessible templating library allows quick sharing of job templates within your deployment only!

Job Templates

Method	Endpoint	Description	CLI Command
GET	`/api/v1/templates`	List job templates	`mjolnir templates list`
POST	`/api/v1/templates`	Create a job template	`mjolnir templates create FILE -n NAME`
GET	`/api/v1/templates/{name}`	Get a template	`mjolnir templates get NAME`
DELETE	`/api/v1/templates/{name}`	Delete a template	`mjolnir templates delete NAME`
POST	`/api/v1/jobs/from-template/{name}`	Create job from template	`mjolnir templates run NAME -n JOB -r REGION`

Webhooks

Method	Endpoint	Description	CLI Command
GET	`/api/v1/webhooks`	List webhooks	`mjolnir webhooks list`
POST	`/api/v1/webhooks`	Register a webhook	`mjolnir webhooks create -u URL -e EVENT`
DELETE	`/api/v1/webhooks/{id}`	Delete a webhook	`mjolnir webhooks delete ID`

Job templates for reusable workflows:

```
gco templates create examples/gpu-job.yaml \
  --name gpu-training \
  -d "GPU training template"
gco templates list
gco templates run gpu-training --name my-run --region us-east-1
```

Options to integrate with external systems at every stage of a job's lifecycle!

Customizable Schedulers
(run multiple in combination or turn them all on)

```
"helm": {
  "keda": { "enabled": true },
  "nvidia_gpu_operator": { "enabled": true },
  "nvidia_dra_driver": { "enabled": true },
  "nvidia_network_operator": { "enabled": true },
  "aws_efa_device_plugin": { "enabled": true },
  "aws_neuron_device_plugin": { "enabled": true },
  "volcano": { "enabled": true },
  "kuberay": { "enabled": true },
  "cert_manager": { "enabled": true },
  "slurm": { "enabled": false },
  "yunikorn": { "enabled": false },
  "kueue": { "enabled": true }
},
```

Schedulers & Orchestrators

Document	Status	Description
Schedulers Overview	—	Comparison, decision guide, and how tools combine
Volcano	Enabled	Gang scheduling and batch job management for distributed training
Kueue	Enabled	Job queueing with resource quotas, fair sharing, and priority
KubeRay	Enabled	Ray distributed computing for training, tuning, and serving
KEDA	Enabled	Event-driven autoscaling from SQS, Prometheus, CloudWatch, and 60+ sources
Slurm (Slinky)	Opt-in	HPC-style scheduling with sbatch/srun on Kubernetes
YuniKorn	Opt-in	App-aware scheduler with hierarchical queues and multi-tenant fair sharing

Multiple Job Submission Types

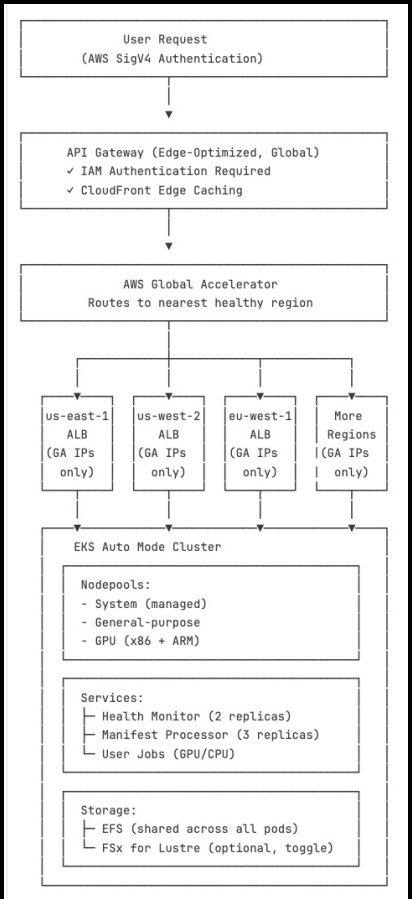
```
# 1. SQS queue (recommended) — async, auto-processed by queue
        processor
gco jobs submit-sqs examples/gpu-job.yaml --region us-east-1

# 2. API Gateway — synchronous, SigV4 authenticated
gco jobs submit examples/gpu-job.yaml -n gco-jobs

# 3. Direct kubectl — requires EKS access entry
gco jobs submit-direct examples/gpu-job.yaml -r us-east-1

# 4. DynamoDB global queue — centralized tracking with status history
gco jobs submit-queue examples/gpu-job.yaml --region us-east-1
```

Security in Depth



Five layers protect every request:

1. **IAM Authentication** — API Gateway validates AWS credentials (SigV4)
2. **Secret Header** — Lambda injects a rotating token from Secrets Manager
3. **IP Restriction** — ALBs only accept Global Accelerator IPs
4. **Header Validation** — Backend services verify the secret token
5. **IRSA** — Pods use IAM roles for AWS access (no static credentials)

```
Request flow: User → API Gateway (SigV4) → Lambda (adds secret) → Global Accelerator
  → ALB (GA IPs only) → Services (validate secret)
```

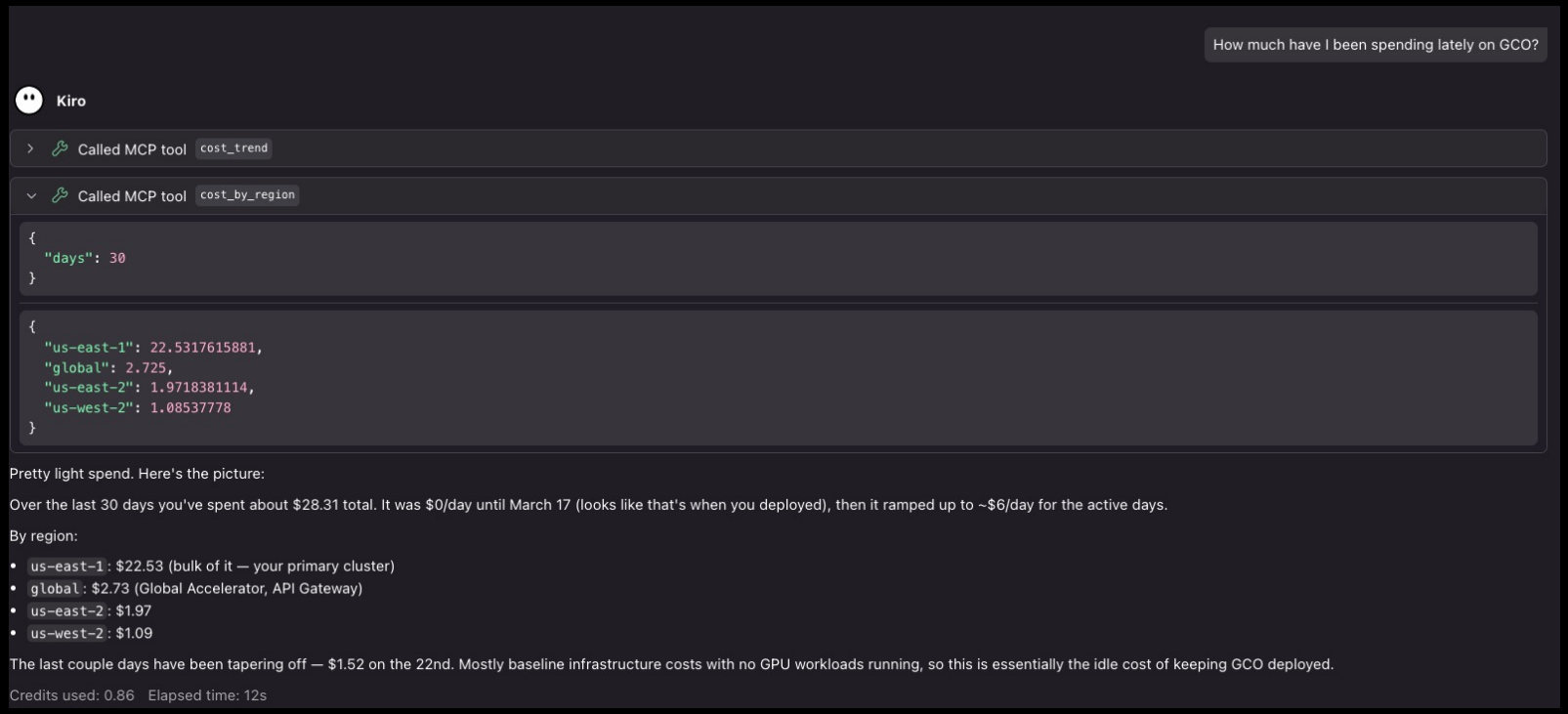
Built-in KEDA Queue Processor

Submit with automatic region selection:

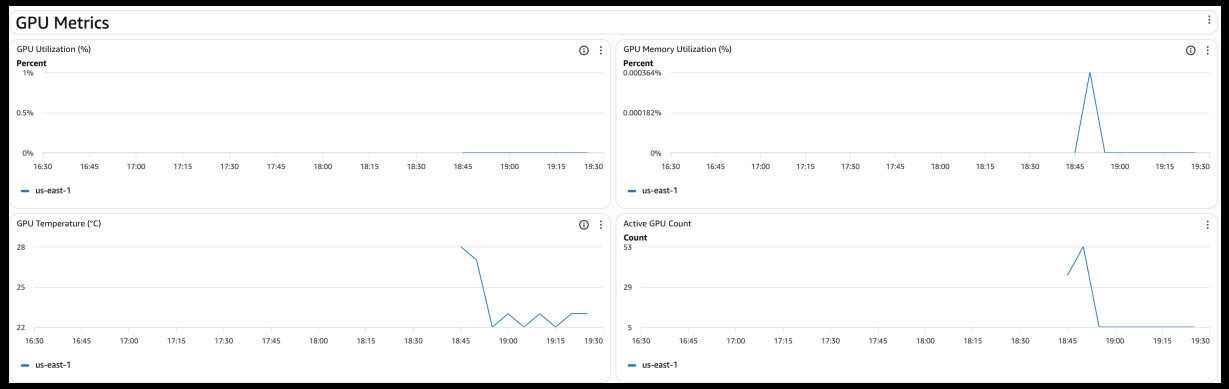
```
gco jobs submit-sqs examples/gpu-job.yaml --auto-region
```

```
User runs: gco jobs submit-sqs manifest.yaml --region us-east-1
  → Manifest sent to regional SQS queue
  → KEDA detects message, spins up consumer pod
  → Consumer applies manifest to cluster
  → Message deleted, pod terminates
  → KEDA scales back to zero when queue is empty
```

In-built Cost Management Tools



Monitoring From a Single Pane of Glass



Metrics from all regions all flow into a single CloudWatch dashboard provisioned by CloudFormation automatically as part of the larger deployment. Also includes several built in alarm and alerting features.

The dashboard is far too large to screenshot here but above are pictures from the GPU metric section (hosted in us-east-2) showing GPU metrics scraped automatically from the cluster using the Nvidia DCGM node exporter in us-east-1

Powerful and Extensive API

GCO Manifest Processor API

This document describes the REST API for the GCO Manifest Processor service.

Table of Contents

Using AWS CLI with SigV4

```
# Using awscurl (recommended)
pip install awscurl

awscurl --service execute-api \
  --region us-east-1 \
  "https://<api-gateway-endpoint>/api/v1/jobs"

# Or using curl with AWS credentials
curl "https://<api-gateway-endpoint>/api/v1/jobs" \
  --aws-sigv4 "aws:amz:us-east-1:execute-api" \
  --user "$AWS_ACCESS_KEY_ID:$AWS_SECRET_ACCESS_KEY"
```

Using Python with boto3

```
import requests
from botocore.auth import SigV4Auth
from botocore.awsrequest import AWSRequest
import boto3

session = boto3.Session()
credentials = session.get_credentials()

request = AWSRequest(method='GET', url='https://<api-gateway-endpoint>/api/v1/jobs')
SigV4Auth(credentials, 'execute-api', 'us-east-1').add_auth(request)

response = requests.get(request.url, headers=dict(request.headers))
```

See Client Examples for more detailed examples.

Capacity Finding Tools

- Analyze live signals to infer available capacity across one or more regions
- Converse with an AI about capacity recommendations powered by live data and your choice of Bedrock model (defaults to Claude Sonnet 4.0)

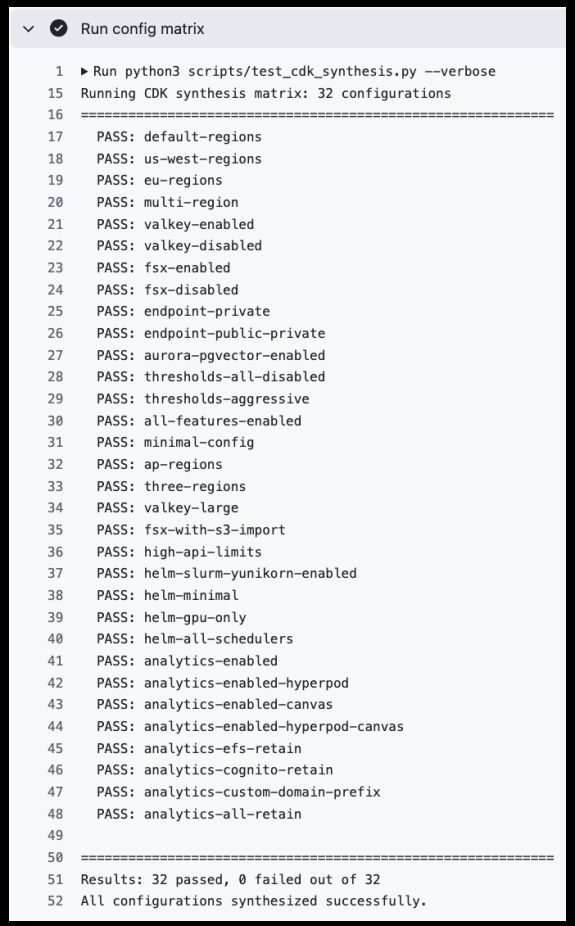
Check GPU availability:

```
gco capacity check --instance-type g5.xlarge --region us-east-1
gco capacity recommend-region --gpu
```

The CLI checks Spot Placement Scores and instance availability across regions to find where GPUs are actually available right now.

```
gco capacity ai-recommend \
  --workload "Training a large language model" \
  --gpu \
  --min-gpus 4
```

Comprehensive CI



- 90%> code coverage
- 9 different security scanners (Bandit, CodeQL, Trivy, Checkov, KCIS, Semgrep, Gitleaks, Trufflehog, pip-audit)
- 9 different linters (CodeQL, Ruff, Flake8, Black, Isort MyPy, Yamllint, Hadolint, ShellCheck)
- Over 5,500 total unit tests (PyTest and BATS)
- CDK Synthesis Matrix test makes sure the project reliably synthesizes CloudFormation under 32 diverse configuration sets.

5 CDK Nag Conformance Packs

Conformance Pack	CDK-Nag Class	What It Checks	Applied In
AWS Solutions	`AwsSolutionsChecks`	AWS architecture best practices (IAM, encryption, logging, networking)	`app.py` — all stacks
HIPAA Security	`HIPAASecurityChecks`	Healthcare data protection rules (encryption at rest/transit, logging, access controls)	`app.py` — all stacks
NIST 800-53 Rev 5	`NIST80053R5Checks`	Federal security controls (access management, audit, system protection)	`app.py` — all stacks
PCI DSS 3.2.1	`PCIDSS321Checks`	Payment card industry standards (encryption, network segmentation, monitoring)	`app.py` — all stacks
Serverless	`ServerlessChecks`	Serverless best practices (Lambda concurrency, DLQ, timeouts)	`app.py` — all stacks

All five packs run on every stack during `cdk synth`. Suppressions for legitimate exceptions (like EKS requiring managed IAM policies, or access log buckets not needing replication) are documented in nag_suppressions.py with explicit reasons. The synth fails if any unsuppressed violation is found — it's strict, not advisory.

app.py 5.24 KiB

```python
#!/usr/bin/env python3
"""
GCO (Global Capacity Orchestrator on AWS) - Multi-Region EKS Auto Mode Platform for AI/ML Workloads

This is the main CDK application entry point that orchestrates the deployment of:
- Global Stack: AWS Global Accelerator for multi-region routing
- API Gateway Stack: Centralized IAM-authenticated entry point
- Regional Stacks: EKS clusters, ALBs, and services per region
- Monitoring Stack: Cross-region CloudWatch dashboards and alarms

Architecture:
    User → API Gateway (IAM Auth) → Global Accelerator → Regional ALB → EKS Services

Usage:
    cdk deploy --all                    # Deploy all stacks
    cdk deploy gco-us-east-1            # Deploy single region
    cdk destroy --all                   # Cleanup all resources
"""

import aws_cdk as cdk
from cdk_nag import (
    AwsSolutionsChecks,
    HIPAASecurityChecks,
    NIST80053R5Checks,
    PCIDSS321Checks,
    ServerlessChecks,
)
```

https://github.com/cdklabs/cdk-nag/blob/main/RULES.md

02

Auto Routing

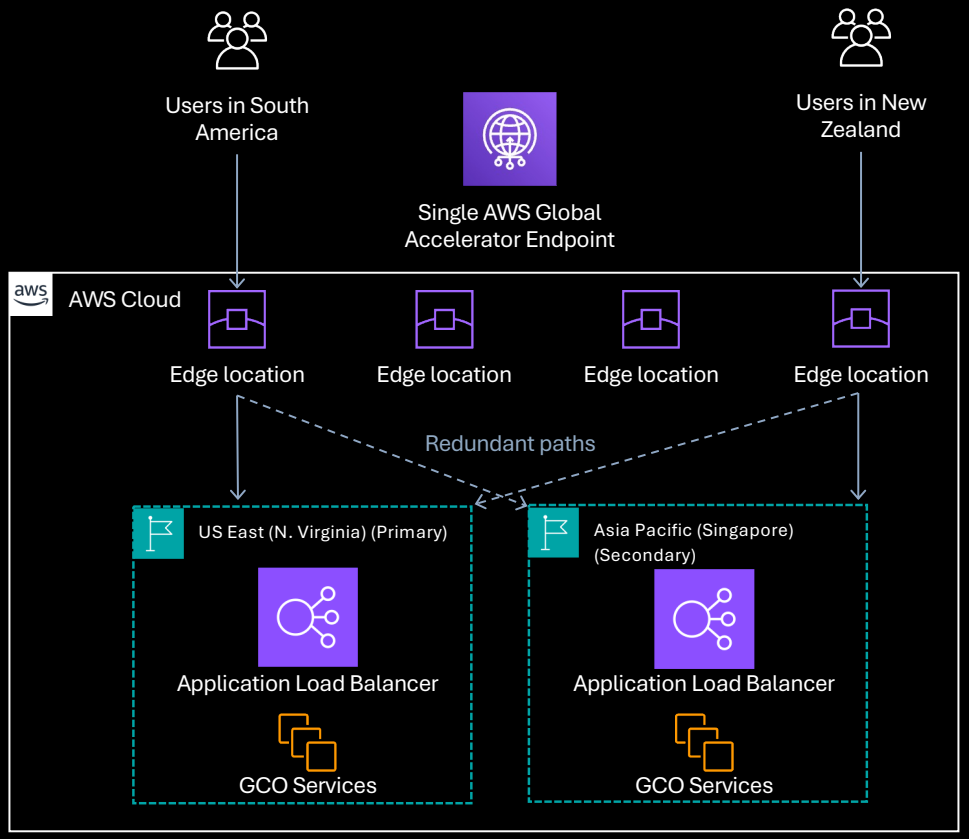
Automatically selecting the best region for workloads

GCO Auto Routing Capability #1:

Take my workload to the nearest region and don't go to a region if it is already running too much stuff.

GCO AWS Global Accelerator Concepts

- USES AWS GLOBAL NETWORK FROM EDGE TO REGION
- CLIENT TRAFFIC INGRESSES VIA CLOSEST AVAILABLE EDGE LOCATION
- ROUTE CLIENT TO CLOSEST HEALTHY ENDPOINT (THIS IS VERY IMPORTANT)



The Problem: Run my job ASAP wherever there's capacity

The Solution: Automated resource-based job routing

```
"resource_thresholds": {
  "cpu_threshold": 60,
  "memory_threshold": 60,
  "gpu_threshold": 60,
  "pending_pods_threshold": 10,
  "pending_requested_cpu_vcpus": 100,
  "pending_requested_memory_gb": 200,
  "pending_requested_gpus": 8
},
```

- Generally, when an application fails health checks its because its not working.
- However, there's no reason we can't *programmatically* fail healthchecks and use this to control Global Acclerator's routing behavior.
- All of these thresholds can be changed at any time in cdk.json and redeployed out to the infra
- If an EKS cluster exceeds any of these thresholds incoming jobs will route away from the cluster to other clusters

GCO Auto Routing Capability #2:

Automatically determine the optimal region for my workload.

The Problem: Knowing what region to run in can be difficult

The Solution: Give customers better information and tools

Check GPU availability:

```
gco capacity check --instance-type g5.xlarge --region us-east-1
gco capacity recommend-region --gpu
```

The CLI checks Spot Placement Scores and instance availability across regions to find where GPUs are actually available right now.

Submit with automatic region selection:

```
gco jobs submit-sqs examples/gpu-job.yaml --auto-region
```

```
gco capacity ai-recommend \
  --workload "Training a large language model" \
  --gpu \
  --min-gpus 4
```

GCO Auto Routing Capability #3:

Set up autoscaling, multi-region, latency minimized custom inference endpoints with automatic failover and IAM authentication using a single CLI command.

The Problem: Setting up multi-region inference is hard

The Solution: Automate the process in one command

Deploy a vLLM endpoint using the default `facebook/opt-125m` model (small, loads in seconds, good for testing). Omitting `-r` deploys to all regions so Global Accelerator routing works consistently.

```
gco inference deploy vllm-demo \
  -i vllm/vllm-openai:v0.17.0 \
  --gpu-count 1 \
  --replicas 1
```

Once the endpoint is running, send a prompt through the API Gateway:

```
gco inference invoke vllm-demo \
  -p "Explain GPU orchestration for ML workloads."
```

The command auto-detects that the image is vLLM and sends an OpenAI-compatible `/v1/completions` request via the API Gateway (SigV4 authenticated). The response text is printed directly.

Deploy a second endpoint with HPA-based autoscaling:

```
gco inference deploy vllm-auto \
  -i vllm/vllm-openai:v0.17.0 \
  --gpu-count 1 \
  --replicas 2 \
  --min-replicas 1 --max-replicas 8 \
  --autoscale-metric cpu:70
```

The inference_monitor creates a Kubernetes HPA that scales between 1 and 8 replicas based on CPU utilization.

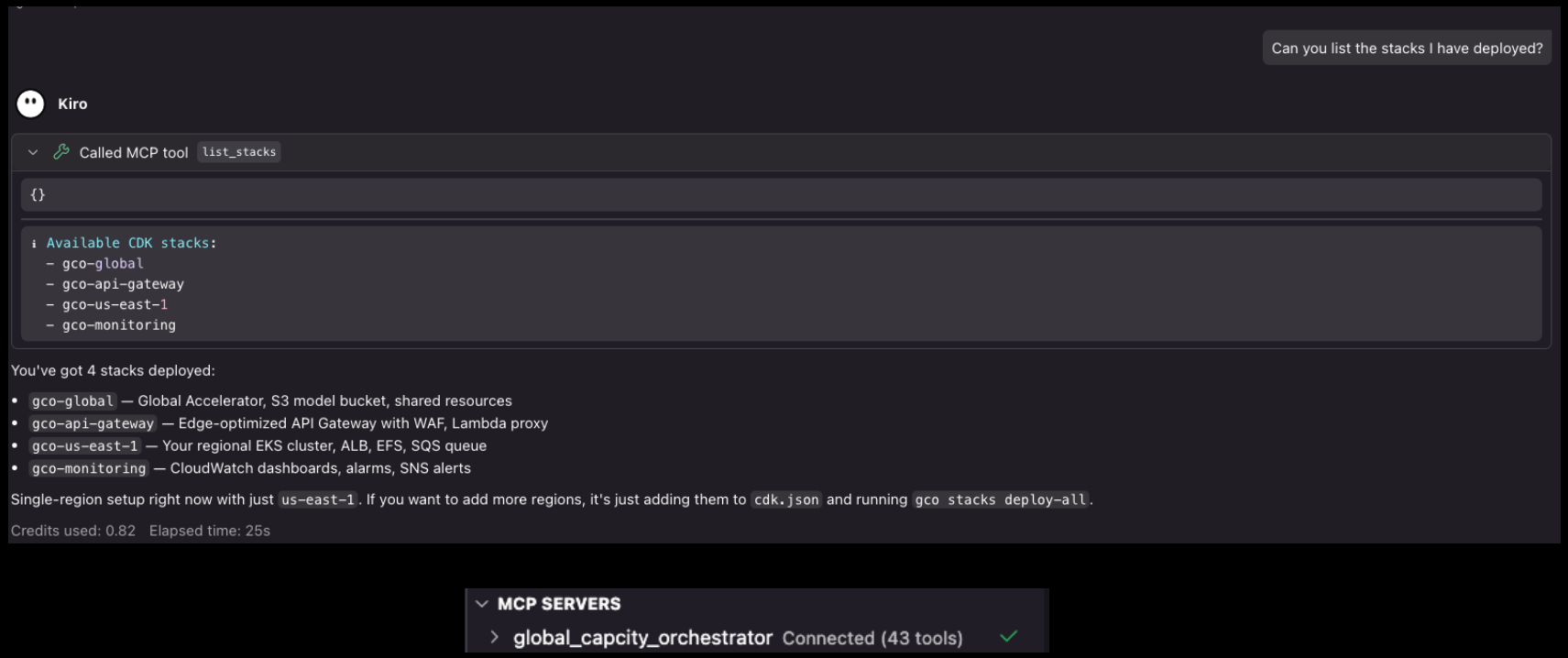
```
gco inference status vllm-auto
```

03

GCO MCP

Access to over 110 different tools that can be used by agents and organized context for all files and documentation in the codebase.

Use the MCP wherever the MCP protocol is supported



Using the MCP in Kiro

Many supported tools

Model Weights

Tool	Description
`list_models`	List uploaded model weights in S3
`get_model_uri`	Get S3 URI for a model

Storage

Tool	Description
`list_storage_contents`	List contents of shared EFS storage
`list_file_systems`	List EFS and FSx file systems

Infrastructure

Tool	Description
`list_stacks`	List all GCO CDK stacks
`stack_status`	Get detailed CloudFormation stack status
`fsx_status`	Check FSx for Lustre configuration

Cost Tracking

Tool	Description
`cost_summary`	Total spend broken down by AWS service
`cost_by_region`	Cost breakdown by AWS region
`cost_trend`	Daily cost trend
`cost_forecast`	Forecast costs for the next N days

Inference Endpoints

Tool	Description
`deploy_inference`	Deploy an inference endpoint across regions
`list_inference_endpoints`	List all inference endpoints
`inference_status`	Get detailed status with per-region breakdown
`scale_inference`	Scale an endpoint's replica count
`update_inference_image`	Rolling update to a new container image
`stop_inference`	Stop an endpoint (scales to zero, keeps config)
`start_inference`	Start a stopped endpoint
`delete_inference`	Delete an endpoint
`canary_deploy`	A/B test a new image version with weighted traffic
`promote_canary`	Promote canary to primary (100% traffic)
`rollback_canary`	Rollback canary (100% traffic to primary)

Capacity

Tool	Description
`check_capacity`	Check spot and on-demand capacity for an instance type
`capacity_status`	View capacity across all deployed regions
`recommend_region`	Get optimal region recommendation (supports instance-type-aware weighted scoring)
`spot_prices`	Get current spot prices for an instance type
`ai_recommend`	Get AI-powered capacity recommendation using Amazon Bedrock
`list_reservations`	List On-Demand Capacity Reservations (ODCRs) across regions
`reservation_check`	Check reservation availability and Capacity Block offerings
`reserve_capacity`	Purchase a Capacity Block offering by ID (supports dry-run)

Job Management

Tool	Description
`list_jobs`	List jobs across GCO clusters (all regions or specific)
`submit_job_sqs`	Submit a job via SQS queue (recommended for production)
`submit_job_api`	Submit a job via API Gateway with SigV4 auth
`get_job`	Get details of a specific job
`get_job_logs`	Get logs from a job
`get_job_events`	Get Kubernetes events for a job (debugging)
`delete_job`	Delete a job
`cluster_health`	Get health status of clusters
`queue_status`	View SQS queue status (pending, in-flight, DLQ)

Make Agents an Expert in the Codebase

Available Resources

Beyond tools, the MCP server exposes documentation and source code as resources. This means an agent can read GCO's docs and code on demand to answer in-depth questions about how the platform works.

Documentation (`docs://`)

Resource	Description
`docs://gco/index`	Browse all available docs and examples
`docs://gco/README`	Project README and quickstart
`docs://gco/docs/{name}`	Any doc by name (ARCHITECTURE, CLI, INFERENCE, etc.)
`docs://gco/examples/{name}`	Example manifests (simple-job, inference-vllm, etc.)

Source Code (`source://`)

Resource	Description
`source://gco/index`	Browse all source files grouped by package
`source://gco/config/{filename}`	Project config files (pyproject.toml, cdk.json, etc.)
`source://gco/file/{path}`	Any source file by relative path

Source code resources cover `gco/`, `cli/`, `lambda/`, and `mcp/`. Build artifacts and caches are filtered out. Path traversal outside the project is blocked.

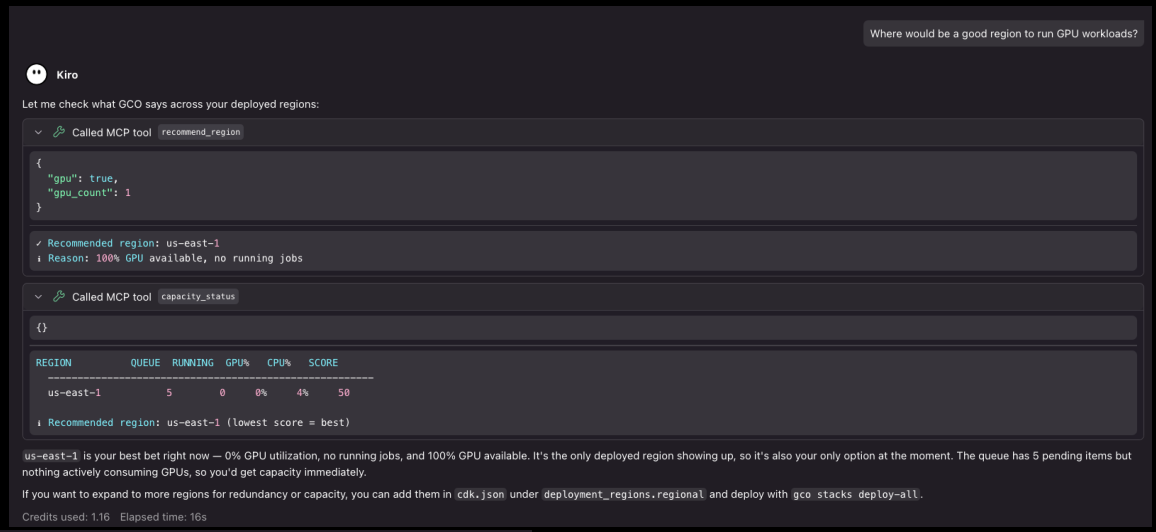
Try it

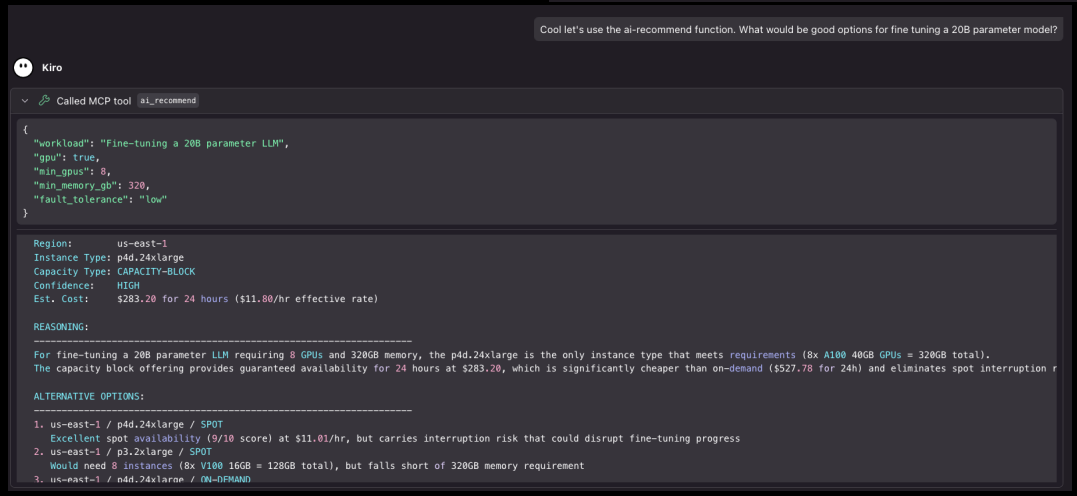
Ask your agent questions like:

- "How does GCO decide which region to recommend for a job?"
- "Walk me through the inference deployment flow"
- "What CDK stacks does GCO create and what's in each one?"
- "How does the manifest processor handle job submissions?"

Capacity Scanning

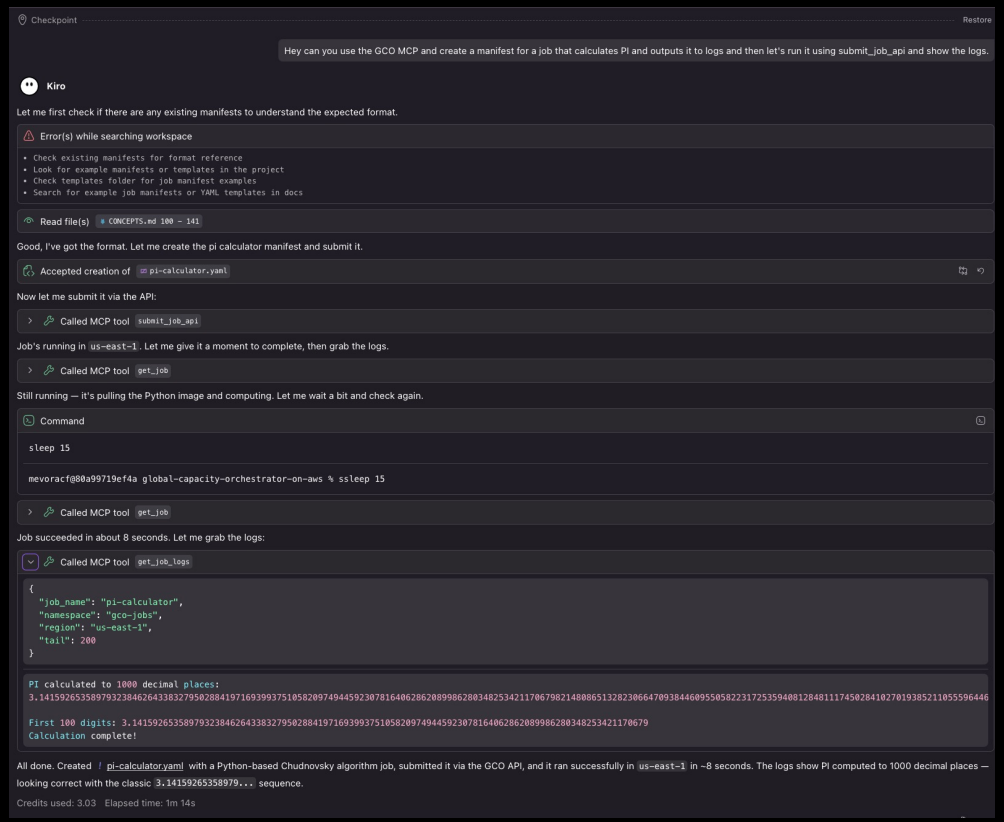
- Solve complex capacity planning problems using nothing but natural language!

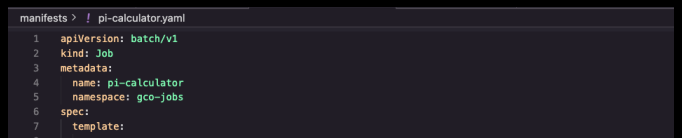




- Combining knowledge of the codebase and capacity is where things get really interesting!

Combining MCP Capabilities to Automate Entire Workflows





Simple Setup and Optional Capacity Purchasing

```
{
  "mcpServers": {
    "gco": {
      "command": "python3",
      "args": ["mcp/run_mcp.py"],
      "cwd": "/path/to/GCO"
    }
  }
}
```

⚠️ **Capacity Block purchasing is disabled by default.** The `reserve_capacity` MCP tool can purchase GPU capacity and incur AWS charges. To enable it, set the environment variable `GCO_ENABLE_CAPACITY_PURCHASE=true` in your MCP server config:

```
{
  "mcpServers": {
    "gco": {
      "command": "python3",
      "args": ["mcp/run_mcp.py"],
      "cwd": "/path/to/GCO",
      "env": {
        "GCO_ENABLE_CAPACITY_PURCHASE": "true"
      }
    }
  }
}
```

Getting Started with the MCP Server

A great way to get familiar with GCO is through the capacity recommendation system. It touches several core concepts — multi-region awareness, GPU capacity, spot pricing, and job scheduling — and gives you a practical feel for how the platform thinks about workload placement.

Try asking:

1. **"Check GPU capacity for g5.xlarge across all regions"** — this calls `check_capacity` and shows you how GCO queries EC2 spot placement scores, spot price history, and on-demand availability.
2. **"Which region should I use for a GPU job?"** — this triggers `recommend_region`, which aggregates queue depth, GPU utilization, and running job counts across all deployed regions, then ranks them. Pass an instance type (e.g. `g5.xlarge`) for weighted multi-signal scoring that also factors in spot placement scores, pricing trends, and capacity block availability.
3. **"Explain how the capacity recommendation works under the hood"** — the agent will read `cli/capacity/` via the source resources and walk you through the three-layer architecture:
   - `CapacityChecker` — core AWS queries (spot scores, pricing, instance offerings)
   - `MultiRegionCapacityChecker` — cross-region aggregation and weighted scoring
   - `BedrockCapacityAdvisor` — optional AI-powered recommendations via Bedrock

From there, you can branch into job submission, inference deployments, or cost tracking — all through natural conversation.

04

Live Demo & FAQ

Thank You

Jacob Mevorach
Senior Generative Artificial Intelligence Solutions Architect
Frontier AI Team

https://www.jacobmevorach.com